Introduction to Spatial Data

Workshop 5 — Module 5

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

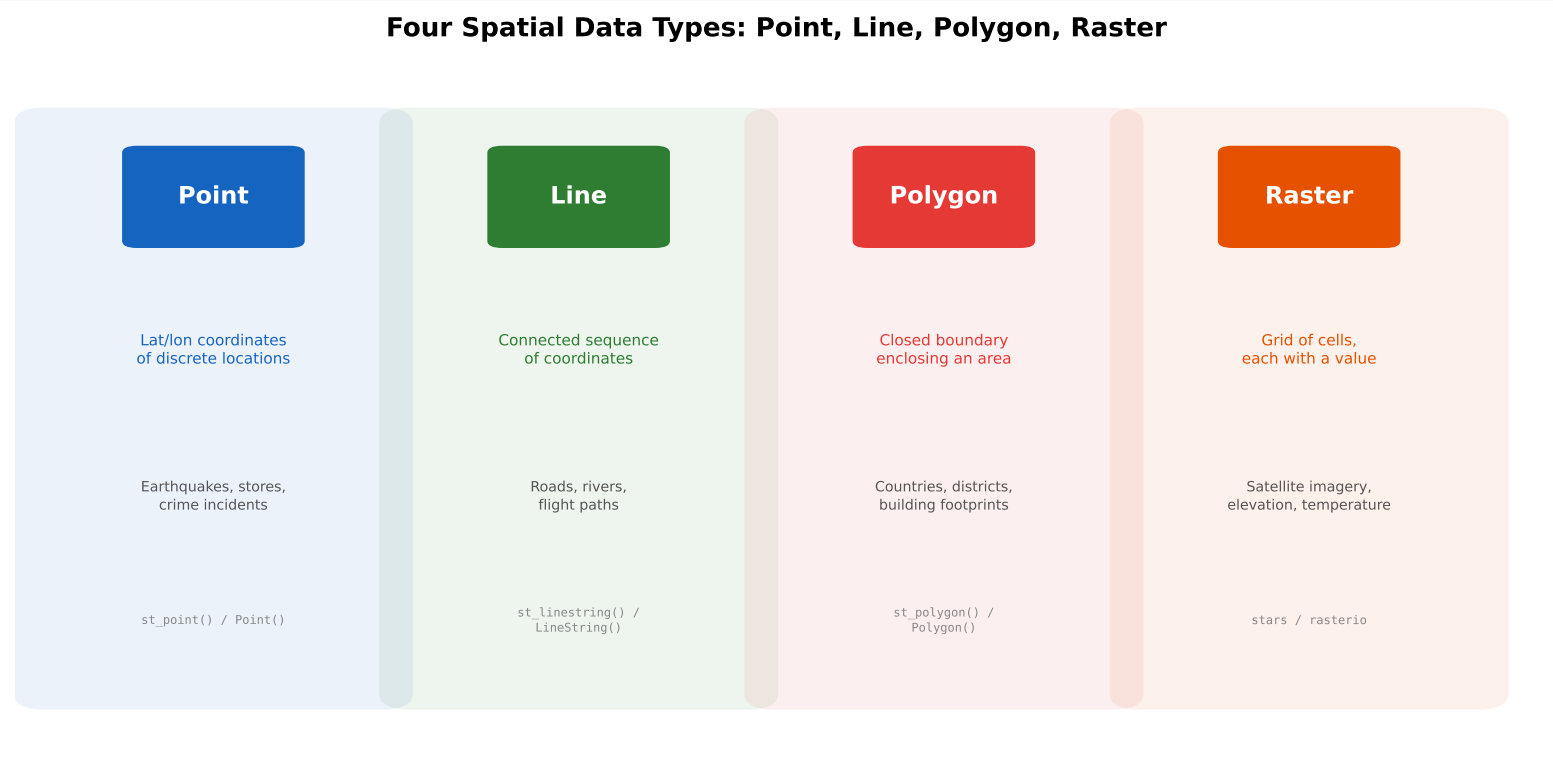
Module Roadmap

1. Four spatial data types: point, line, polygon, raster
2. Coordinate reference systems: geographic vs projected
3. File formats: GeoJSON, Shapefile, GeoPackage
4. The sf/geopandas ecosystem
5. From CSV to map: converting tabular data to spatial

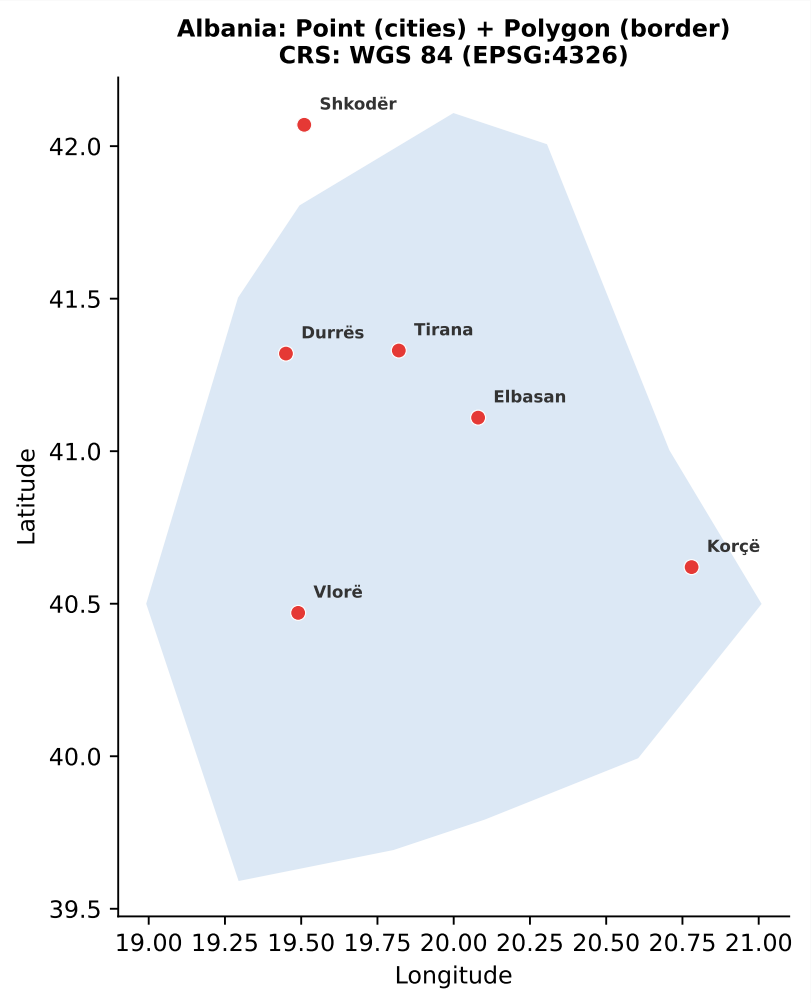
Learning Outcome
You will distinguish geographic from projected CRS, read spatial files in R (sf) and Python (geopandas), and convert a CSV with lat/lon columns into a mappable spatial object.

Spatial Data Types

Point, Line, Polygon, Raster



Albania: Points + Polygon



Data Insight

Cities are **points** (lat/lon pairs). The country border is a **polygon** (closed ring of coordinates). Both reference the same CRS (WGS 84, EPSG:4326). Every spatial operation requires matching CRS between

Coordinate Reference Systems

Geographic vs Projected

Coordinate Reference Systems (CRS): Geographic vs Projected

CRS	Type	Used By	Note
WGS 84 (EPSG:4326)	Geographic (lat/lon degrees)	GPS, web maps, leaflet/folium default	Global standard; units = degrees
Web Mercator (EPSG:3857)	Projected (metres)	Google Maps, OSM tile servers	Distorts area near poles
UTM Zone 34N (EPSG:32634)	Projected (metres)	Albania, local analysis distance calculations	Accurate area/distance for ±3° longitude
Albanian Grid (EPSG:6870)	National (metres)	Official Albanian cadastral data	Local precision; not for global use

Pro-Tip

Use **EPSG:4326** (lat/lon) for storage and web maps. Transform to a **projected CRS** (UTM) before computing distances or areas. In R: `st_transform(sf, 32634)`. In Python: `gdf.to_crs(epsg=32634)`.

File Formats

GeoJSON, Shapefile, GeoPackage

Spatial File Formats: GeoJSON, Shapefile, GeoPackage

Format	Extension	Storage	Notes
GeoJSON	`.geojson`	Text (JSON)	Web-friendly, GitHub renders st_read() / gpd.read_file()
Shapefile	`.shp + .dbf + .shx + .prj`	Binary (4+ files)	Legacy standard, ESRI st_read() / gpd.read_file()
GeoPackage	`.gpkg`	SQLite database	Modern replacement for shapefile st_read() / gpd.read_file()
CSV + coords	`lat, lon columns`	Text	Simplest; needs st_as_sf() or gpd.GeoDataFrame(geometry=)

The Ecosystem

Spatial Packages: R & Python

Spatial Visualization Packages: R & Python

Package	Lang	Key Functions	Notes
`sf`	R	`st_read(), st_transform(), ggplot + geom_sf()`	Modern standard; tidy-compatible replaces sp/rgdal
`tmap`	R	`tm_shape() + tm_polygons()` `tm_dots(), tmap_mode('view')`	Grammar of maps; static + interactive switch with tmap_mode()
`geopandas`	Python	`gpd.read_file(), .plot(), .to_crs()`	pandas + shapely + fiona Core spatial Python
`folium`	Python	`folium.Map() + Choropleth()` `+ Marker(), CircleMarker()`	Leaflet.js wrapper Interactive web maps

Reading Spatial Data

R (sf)

```r
library(sf)
# Read any spatial format
alb <- st_read("albania.geojson")
st_crs(alb)  # check CRS
# Transform to UTM
alb_utm <- st_transform(alb,
    32634)
# Plot with ggplot
ggplot(alb) + geom_sf(fill="#
    E3F2FD") +
  theme_minimal()
# CSV -> spatial
cities <- read_csv("cities.csv")
    |>
  st_as_sf(coords=c("lon","lat"),
           crs=4326)
```

Python (geopandas)

```python
import geopandas as gpd
# Read any spatial format
alb = gpd.read_file("albania.
    geojson")
alb.crs  # check CRS
# Transform
alb_utm = alb.to_crs(epsg=32634)
# Plot
alb.plot(color="#E3F2FD",
    edgecolor="#1565C0")
# CSV -> spatial
from shapely.geometry import
    Point
cities["geometry"] = cities.apply
    (
    lambda r: Point(r.lon, r.lat)
        ,
    axis=1)
gdf = gpd.GeoDataFrame(cities,
    crs=4326)
```

tmap: Grammar of Maps (R)

```r
library(tmap)
# Static map
tm_shape(alb) +
  tm_polygons(col = "population", palette = "Blues", title = "Pop.")
      +
  tm_layout(title = "Albania: Population by District")
# Switch to interactive
tmap_mode("view")  # now all tmap calls produce leaflet maps
tm_shape(alb) + tm_polygons(col = "population") + tm_dots(cities)
# Switch back
tmap_mode("plot")
```

Pro-Tip

tmap is the spatial equivalent of ggplot2: layered grammar (`tm_shape + tm_polygons + tm_dots`). One command (`tmap_mode("view")`) switches between static PDF and interactive leaflet.

Synthesis

Key Takeaways

1. Four spatial types: **point** (locations), **line** (paths), **polygon** (areas), **raster** (grids).
2. **CRS matters**: geographic (degrees) for storage, projected (metres) for distance/area. Always check and match CRS.
3. **GeoJSON** for web, **GeoPackage** for modern work, **Shapefile** for legacy. All readable by `st_read()` and `gpd.read_file()`.
4. **sf** (R) and **geopandas** (Python) are the core packages. **tmap** (R) and **folium** (Python) add map-specific grammar.
5. CSV with lat/lon columns $\rightarrow$ spatial via `st_as_sf()` or `gpd.GeoDataFrame(geometry=Point())`.

Essential Readings

- Lovelace, R. et al. (2019). *Geocomputation with R*. CRC Press. `https://geocompr.robinlovelace.net`
- Rey, S. et al. (2020). *Geographic Data Science with Python*. `https://geographicdata.science`
- sf documentation: `https://r-spatial.github.io/sf/`

Next Module: Choropleth Maps (W05-M06)